Introduction to Intensive Programming in C++

Lecture 9 : Computational Geometry

Elias TSIGARIDAS

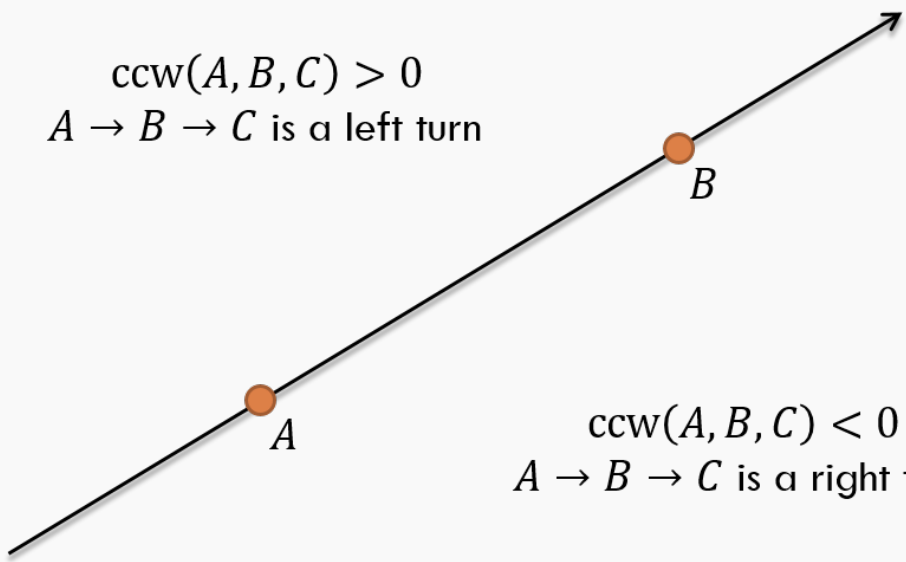
École Polytechnique

Cross Product

Define

$$\text{ccw}(A,B,C) = (B-A)\times(C-A) = (b_x-a_x)(c_y-a_y)-(b_y-a_y)(c_x-a_x)$$

$\text{ccw}(A,B,C) > 0$
$A \to B \to C$ is a left turn



$\text{ccw}(A,B,C) < 0$
$A \to B \to C$ is a right turn

$$\begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix}$$

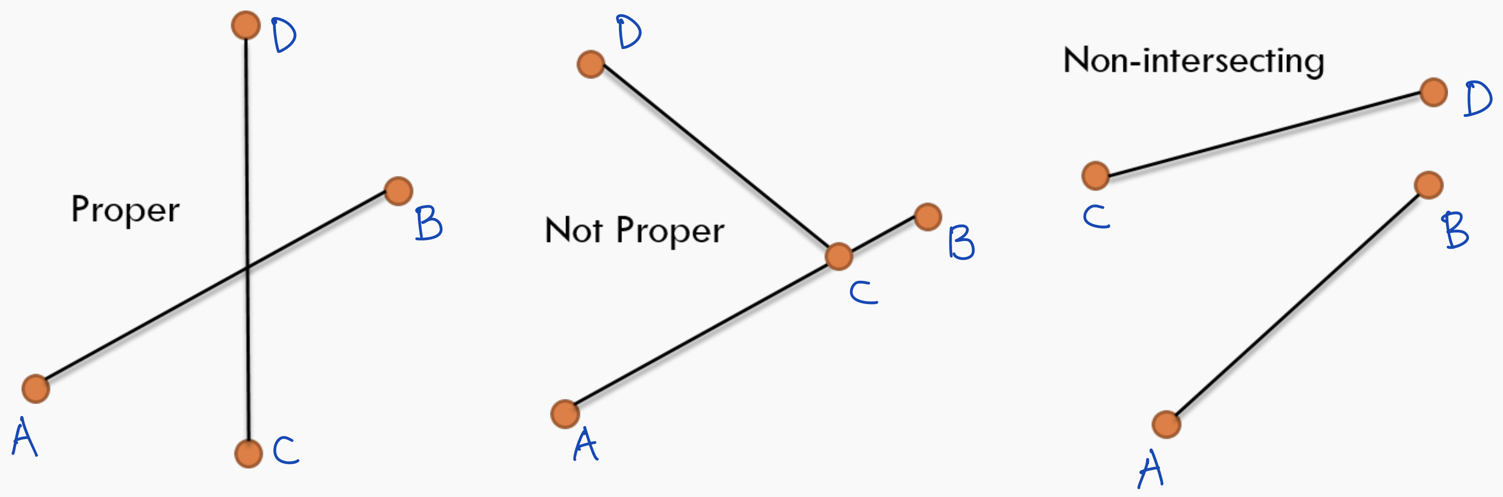
there is a similar test
for high dimensions
what is the size of the matrix?

Applications

- Determining the (signed) area of a triangle
- Testing if three points are collinear
- Determining the orientation of three points
- Testing if two line segments intersect

In most of the cases
we only need the sign

Segment-Segment (proper) Intersection Test



- Assume that the segments intersect
  - From A's point of view, looking straight to B, C and D must lie on different sides
  - Holds true for the other segment as well
- The intersection exists and is proper if:
  - $\operatorname{ccw}(A, B, C) \times \operatorname{ccw}(A, B, D) < 0$
  - *and* $\operatorname{ccw}(C, D, A) \times \operatorname{ccw}(C, D, B) < 0$
- Non-proper intersection!
- If $\operatorname{ccw}(A, B, C)$, $\operatorname{ccw}(A, B, D)$, $\operatorname{ccw}(C, D, A)$, $\operatorname{ccw}(C, D, B)$ are all zeros, then two segments are collinear
- Very careful implementation is required

Contents

Cross Product

Algorithms in Computational Geometry

Polygons

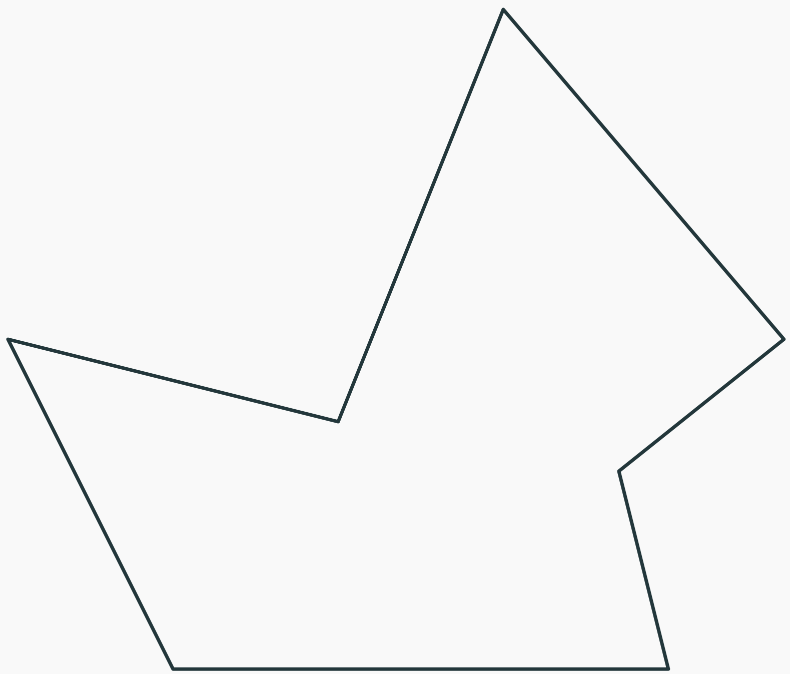
Convex hull

Point in convex polygon

Geometric Binary Search

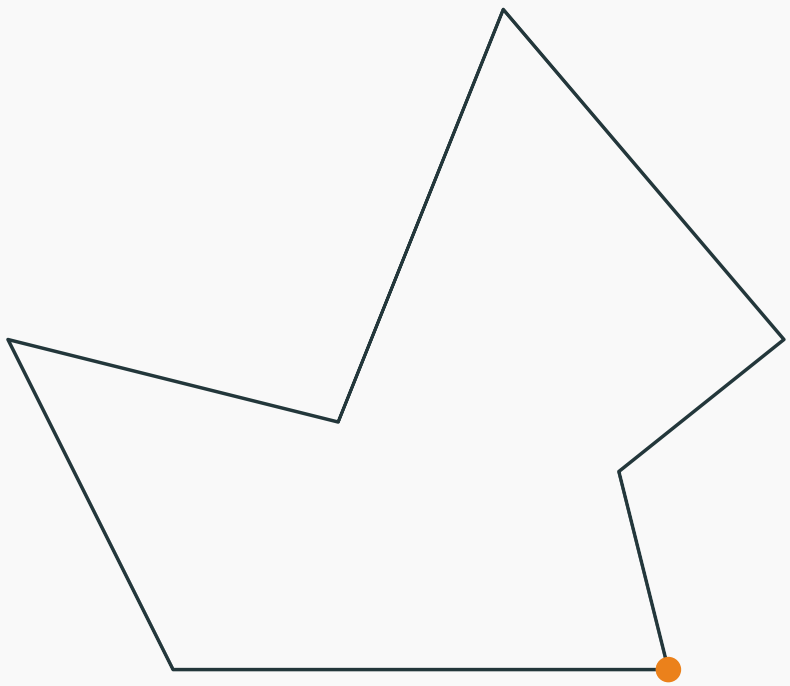
ccw dependent

Polygons and area



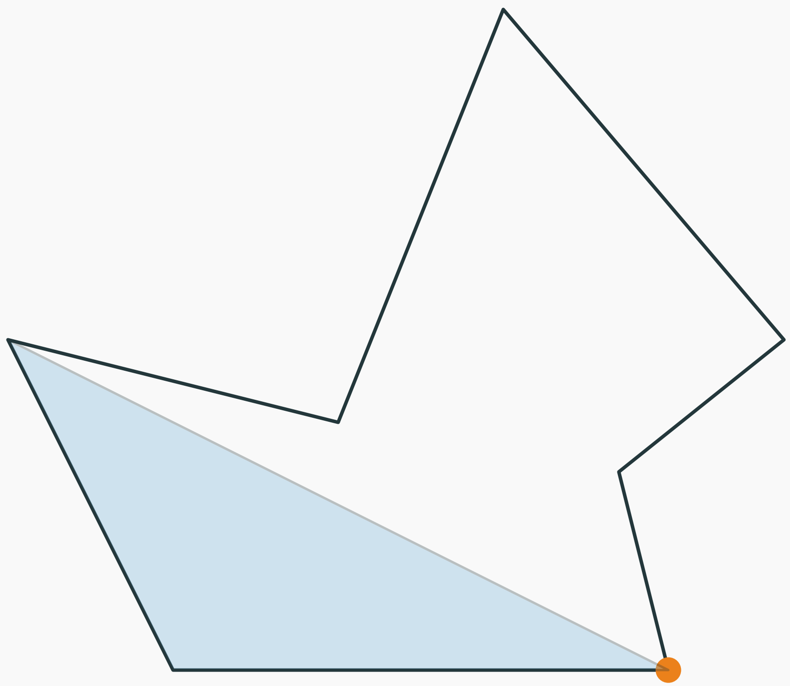
- Represent Polygons using a list of points (in order)

Polygons and area



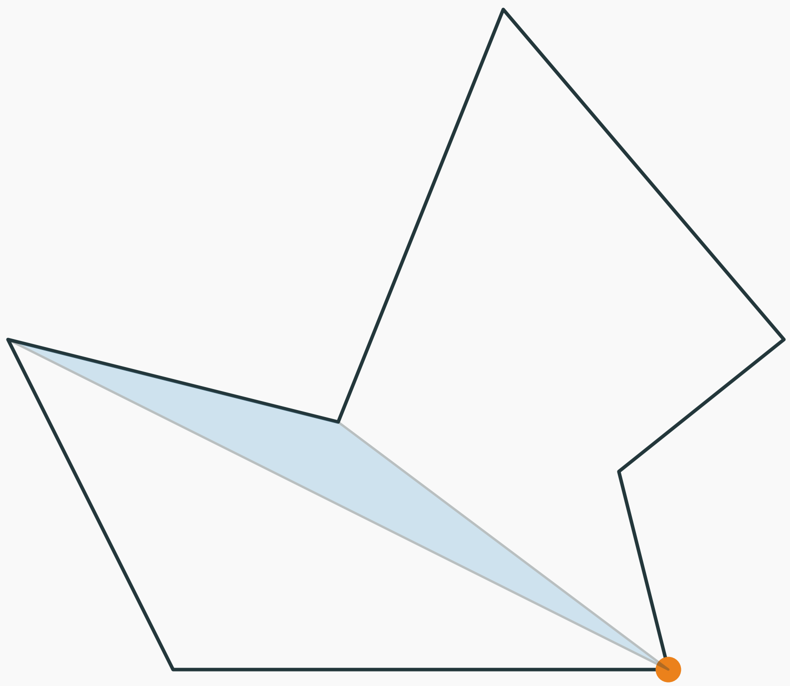
- Represent Polygons using a list of points (in order)
- Calculate the area
  - Pick one starting point.
  - Go through all the other adjacent pair of points and sum the area of the triangulation.
  - Do not worry if you sum up area outside the polygon.

Polygons and area



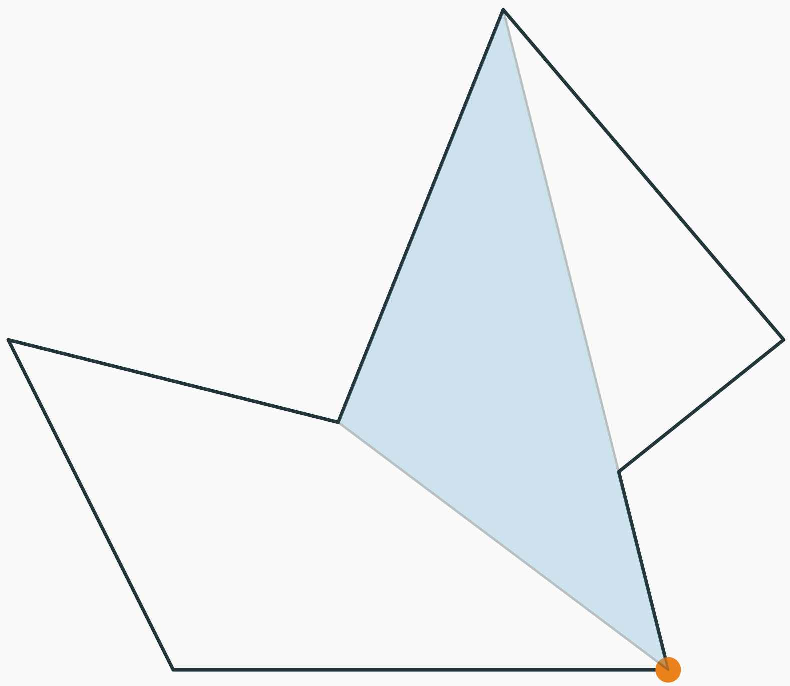
- Represent Polygons using a list of points (in order)
- Calculate the area
  - Pick one starting point.
  - Go through all the other adjacent pair of points and sum the area of the triangulation.
  - Do not worry if you sum up area outside the polygon.

Polygons and area



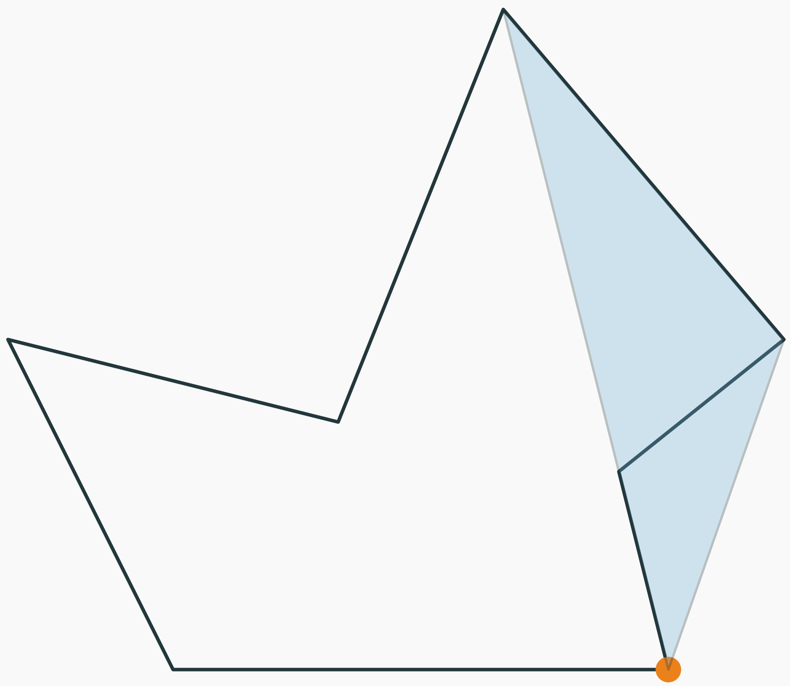
- Represent Polygons using a list of points (in order)
- Calculate the area
  - Pick one starting point.
  - Go through all the other adjacent pair of points and sum the area of the triangulation.
  - Do not worry if you sum up area outside the polygon.

Polygons and area



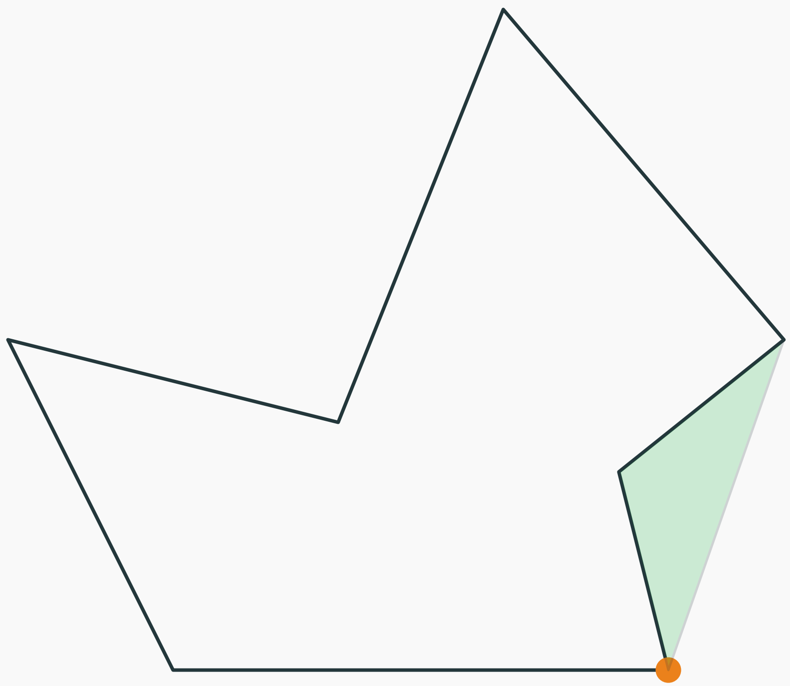
- Represent Polygons using a list of points (in order)
- Calculate the area
  - Pick one starting point.
  - Go through all the other adjacent pair of points and sum the area of the triangulation.
  - Do not worry if you sum up area outside the polygon.

Polygons and area



- Represent Polygons using a list of points (in order)
- Calculate the area
    - Pick one starting point.
    - Go through all the other adjacent pair of points and sum the area of the triangulation.
    - Do not worry if you sum up area outside the polygon.

Polygons and area

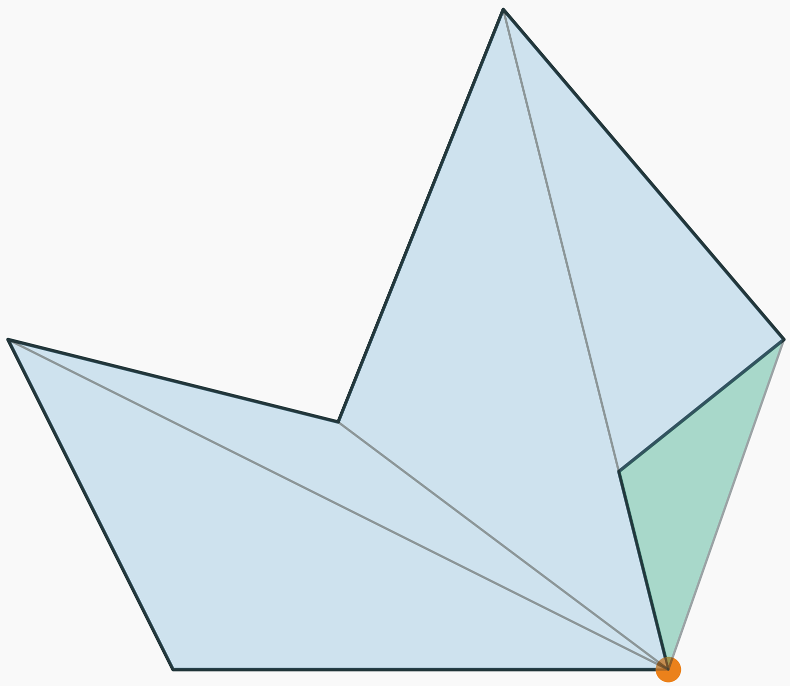


- Represent Polygons using a list of points (in order)
- Calculate the area
  - Pick one starting point.
  - Go through all the other adjacent pair of points and sum the area of the triangulation.
  - Do not worry if you sum up area outside the polygon.

Q: How do you compute the area?

Polygons and area

Compute the signed area with CCW

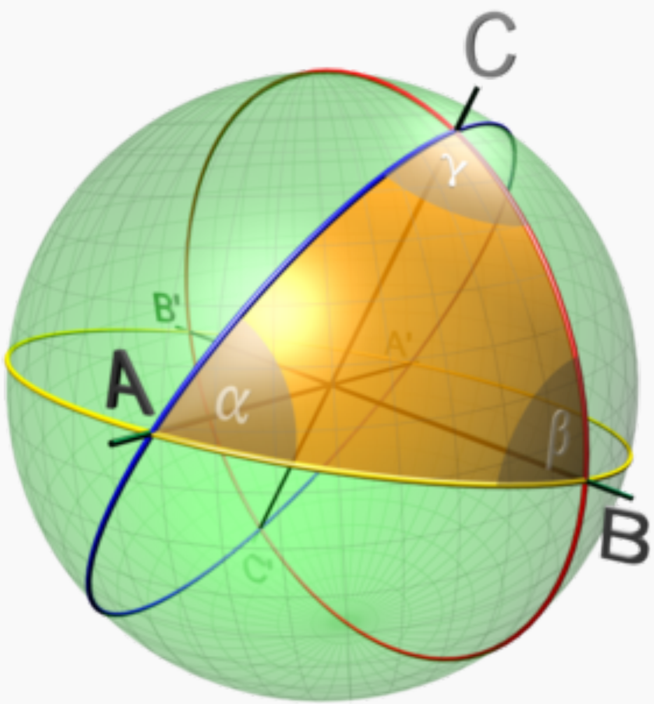


- Represent Polygons using a list of points (in order)
- Calculate the area
  - Pick one starting point.
  - Go through all the other adjacent pair of points and sum the area of the triangulation.
  - Do not worry if you sum up area outside the polygon.

Very simple code.

```
double polygon_area_signed(const vector<point> &p) {
    double area = 0;
    int cnt = size(p);
    for (int i = 1; i + 1 < cnt; i++){
        area += cross(p[i] - p[0], p[i + 1] - p[0])/2;
    }
    return area;
}
double polygon_area(vector<point> &p) {
    return abs(polygon_area_signed(p));
}
```

Polygons on a (unit) sphere



Q: Shortest distance between two points?

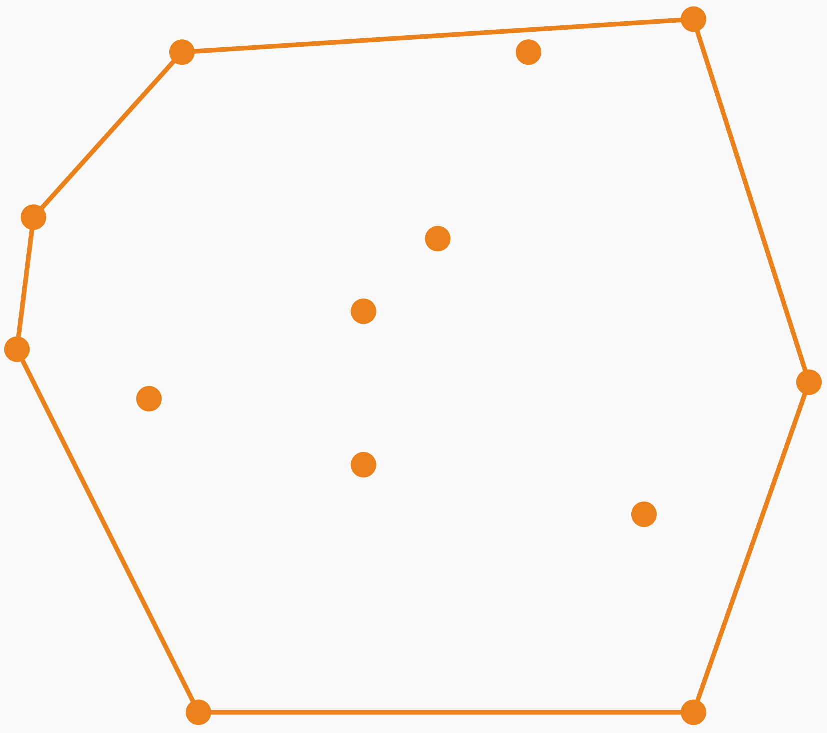
Q: Intersection of 2 "segments"?

$$\text{Area} = \alpha + \beta + \gamma - \pi$$

Area of n-gon $= \alpha_1 + \alpha_2 + \ldots + \alpha_n - (n-2)\pi$?

Convex Hull

- Given a set of points, we want to find the convex hull of the points.

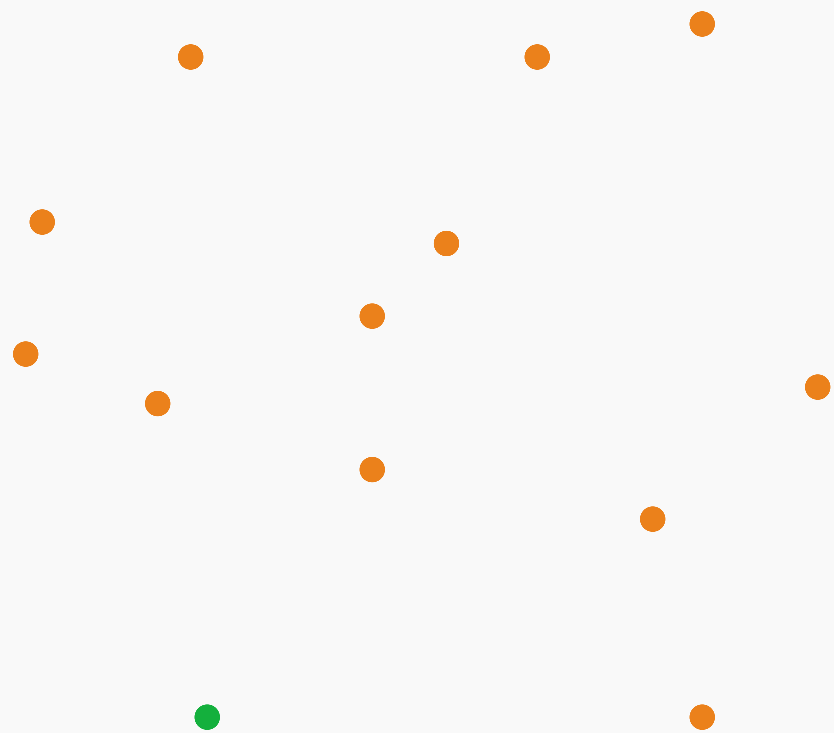


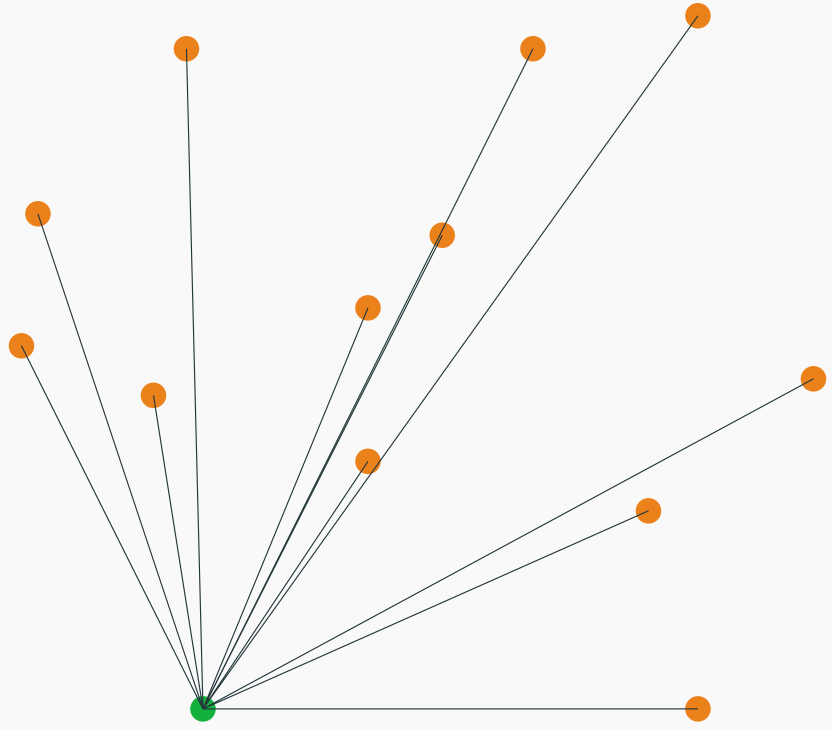
Graham's algorithm

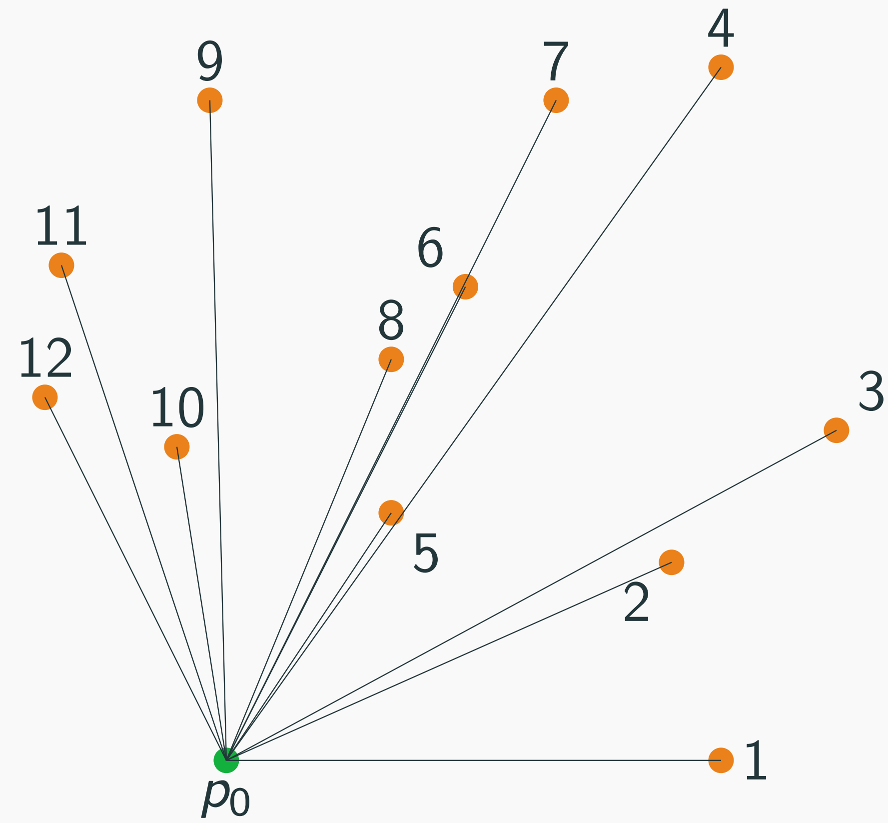
Graham scan

- Pick the point p_0 with the lowest y coordinate.
- Sort all the points by polar angle with p_0.
- Iterate through all the points
- If the current point forms a clockwise angle with the last two points, remove last point from the convex set.
- Otherwise, add the current point to the convex set.

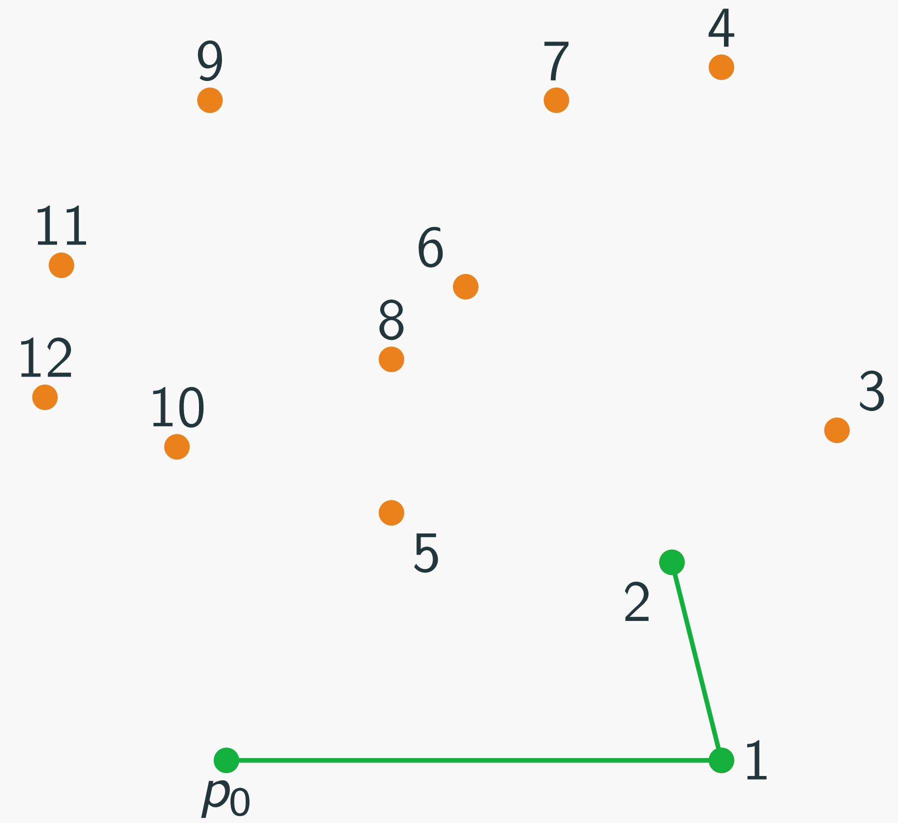
Time complexity $O(N \log N)$.

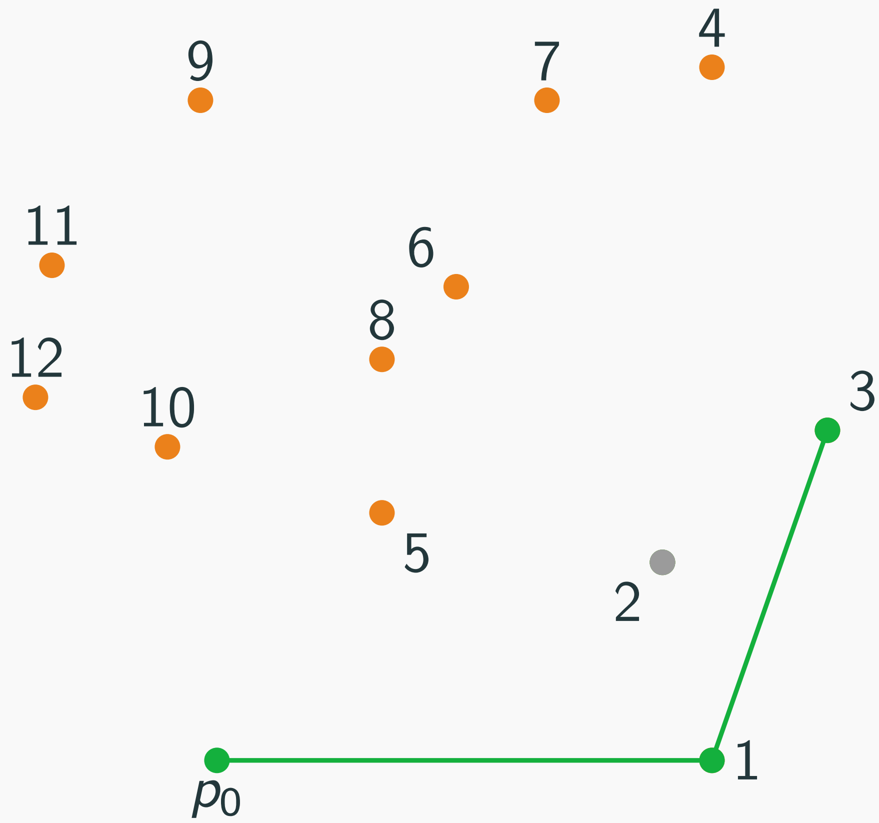




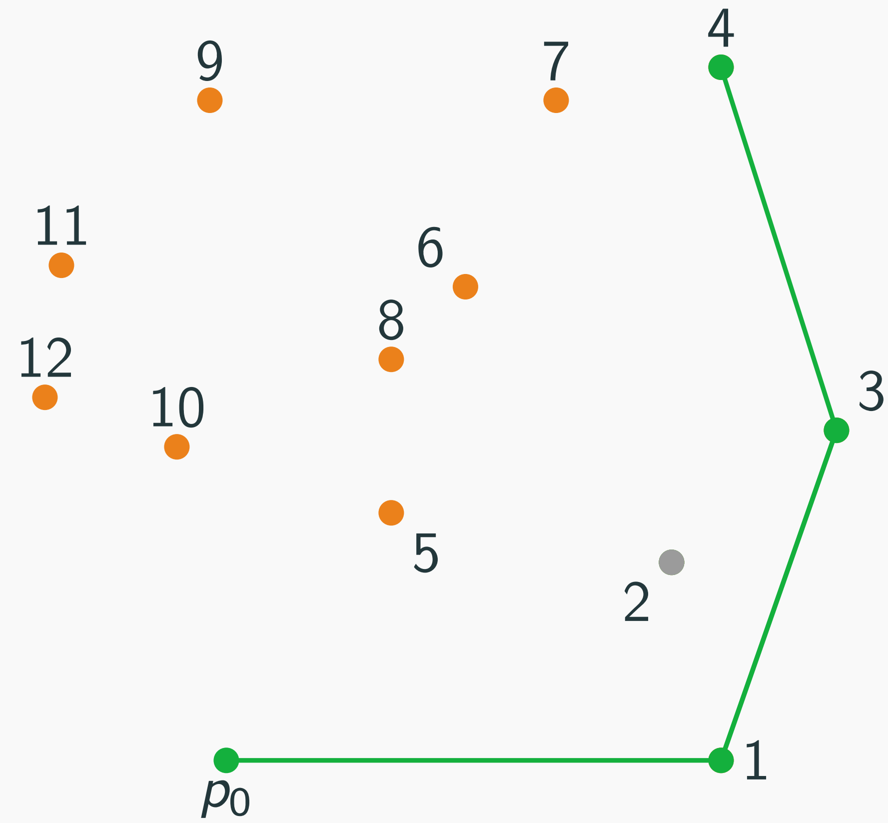


4
9
7
11
6
8
12
10
3
5
2
1
p_0

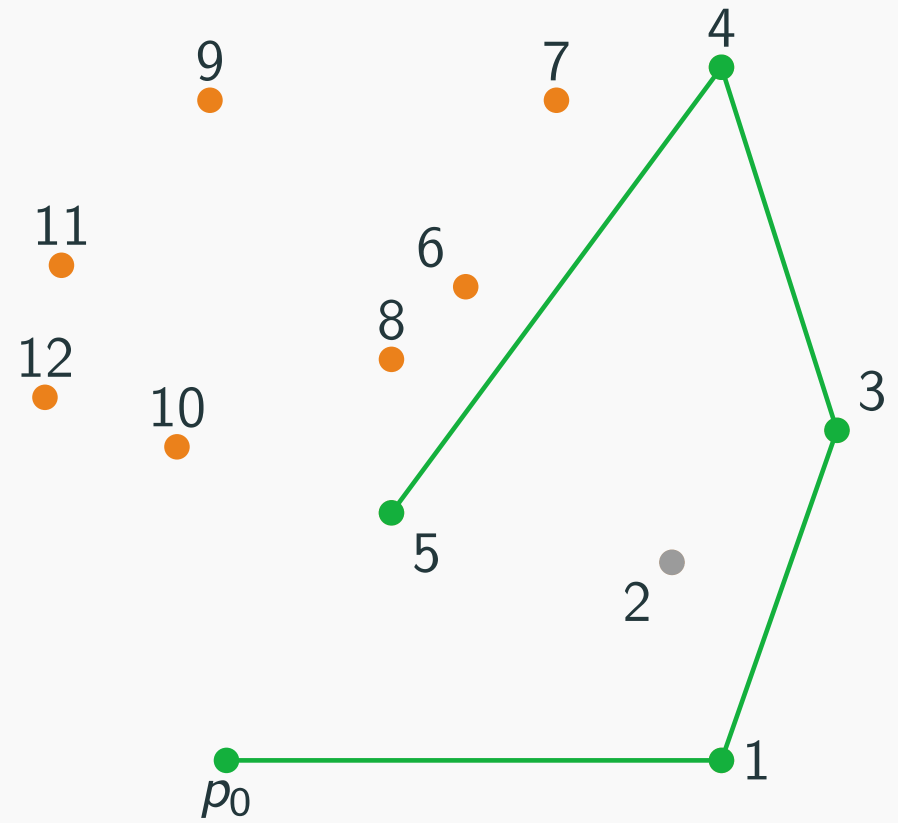

4
9
7
11
6
8
12
3
10
5
2
1
p_0



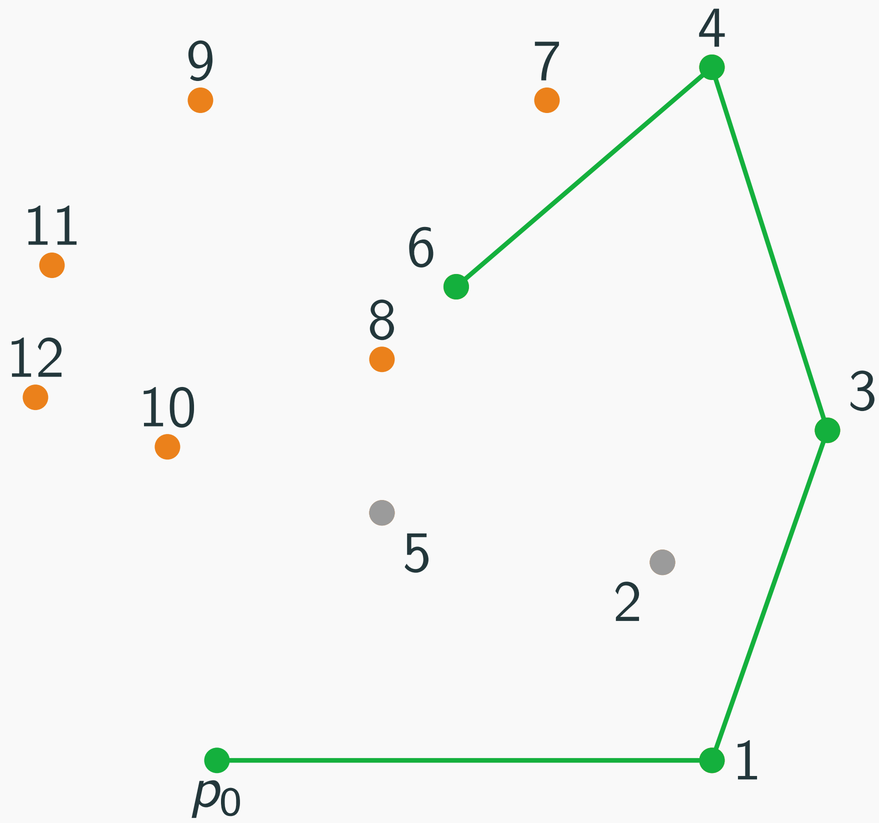
9
7
4
11
6
8
12
10
3
5
2
1
p_0



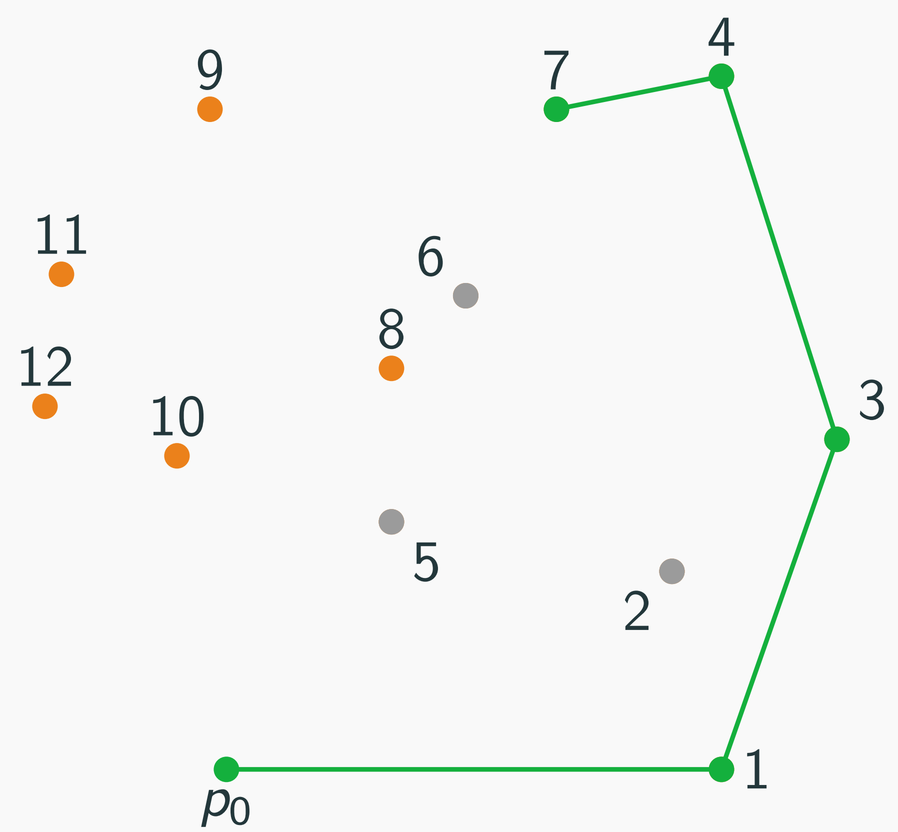
4
9
7
11
6
8
12
3
10
5
2
1
p_0



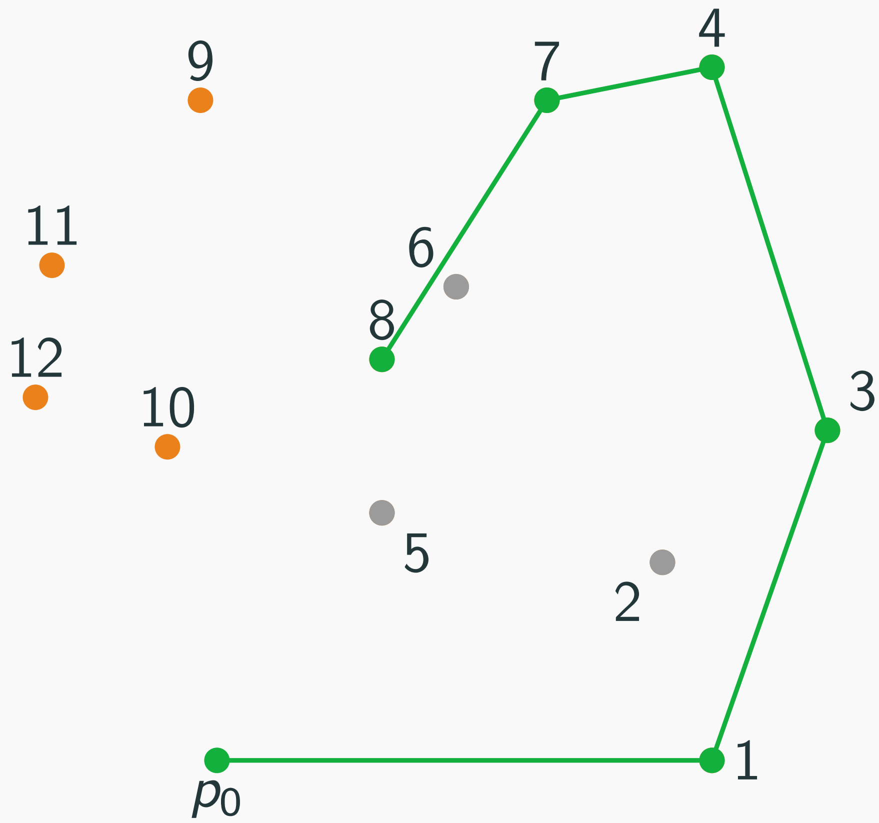
9
7
4
11
6
8
12
10
3
5
2
1
p_0



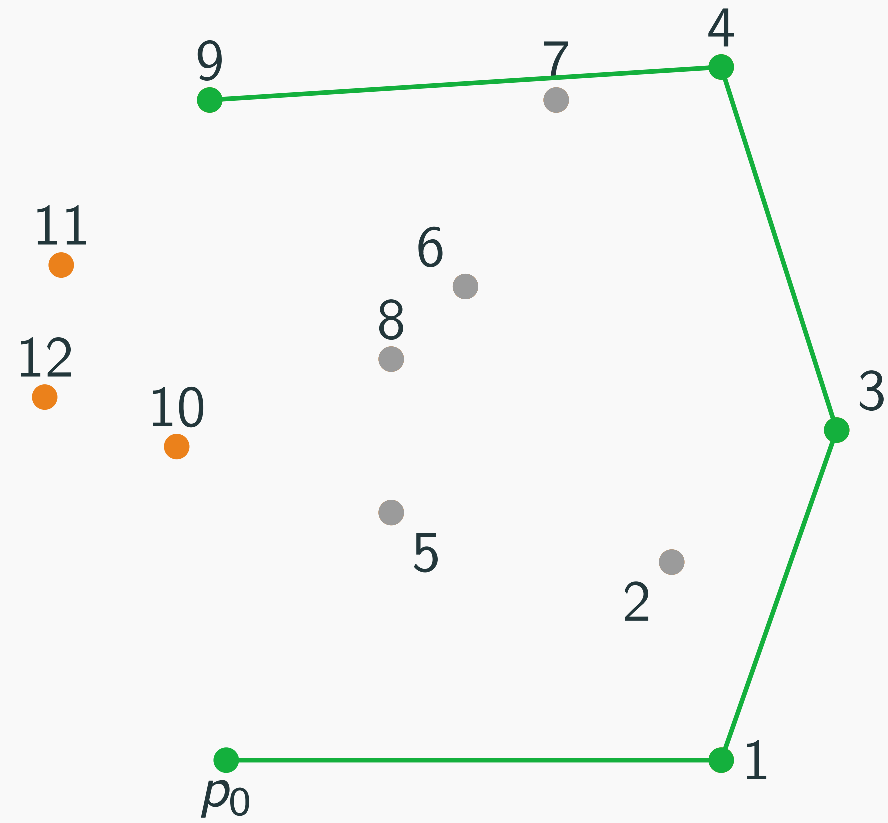
4
9
7
11
6
8
12
3
10
5
2
1
p_0

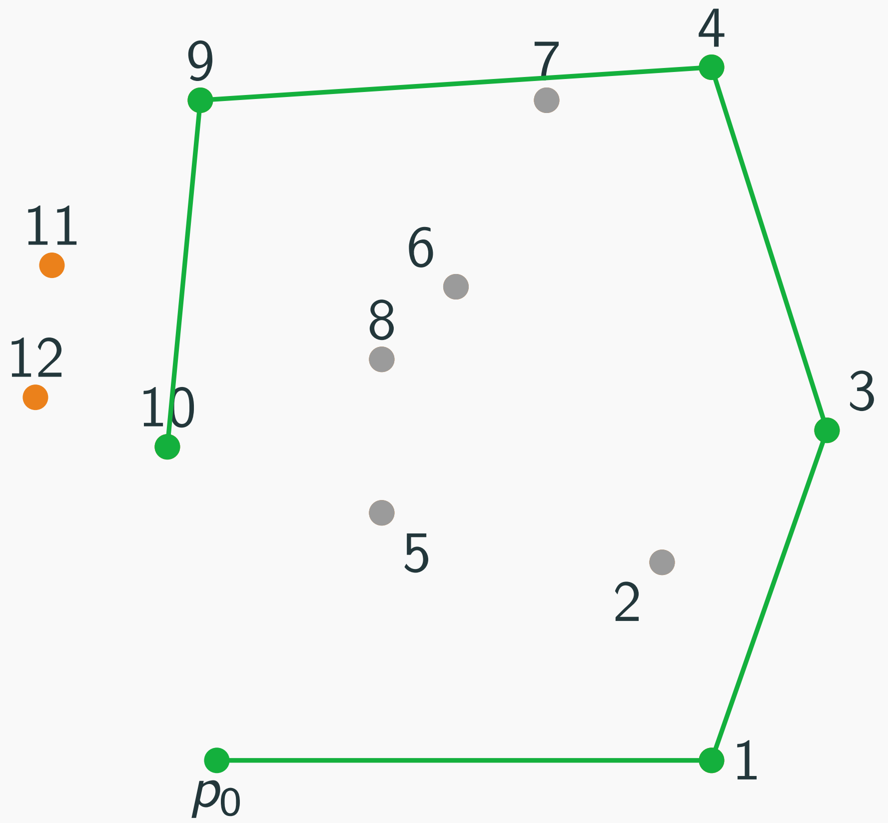


9
7
4
11
6
8
12
3
10
5
2
p_0
1

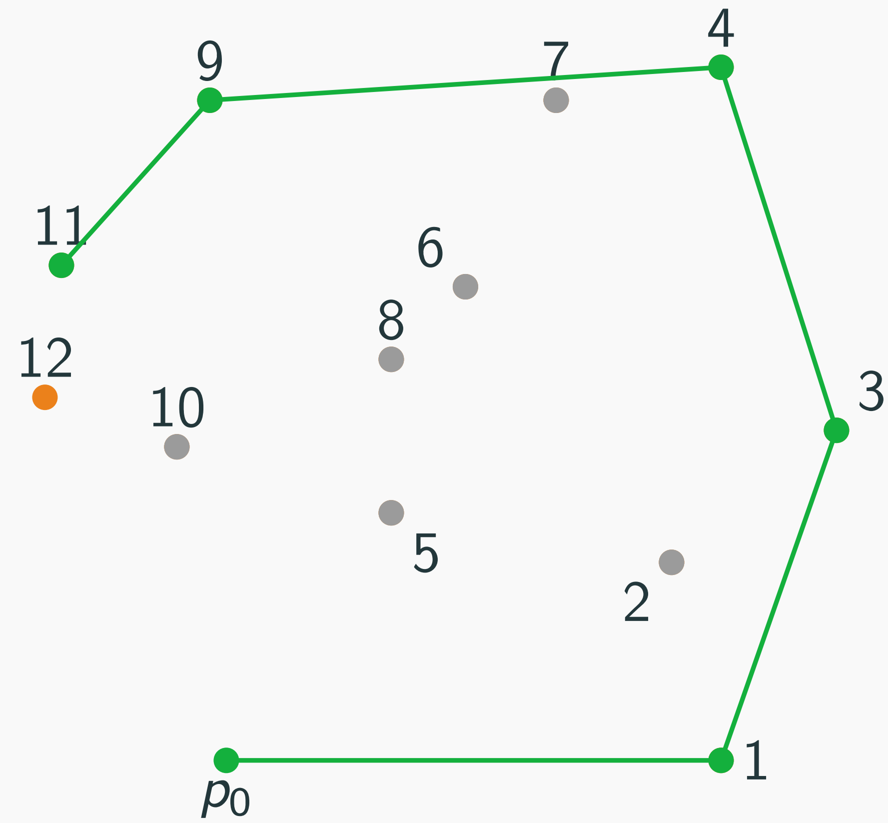


9
7
4
11
6
8
12
3
10
5
2
1
p_0

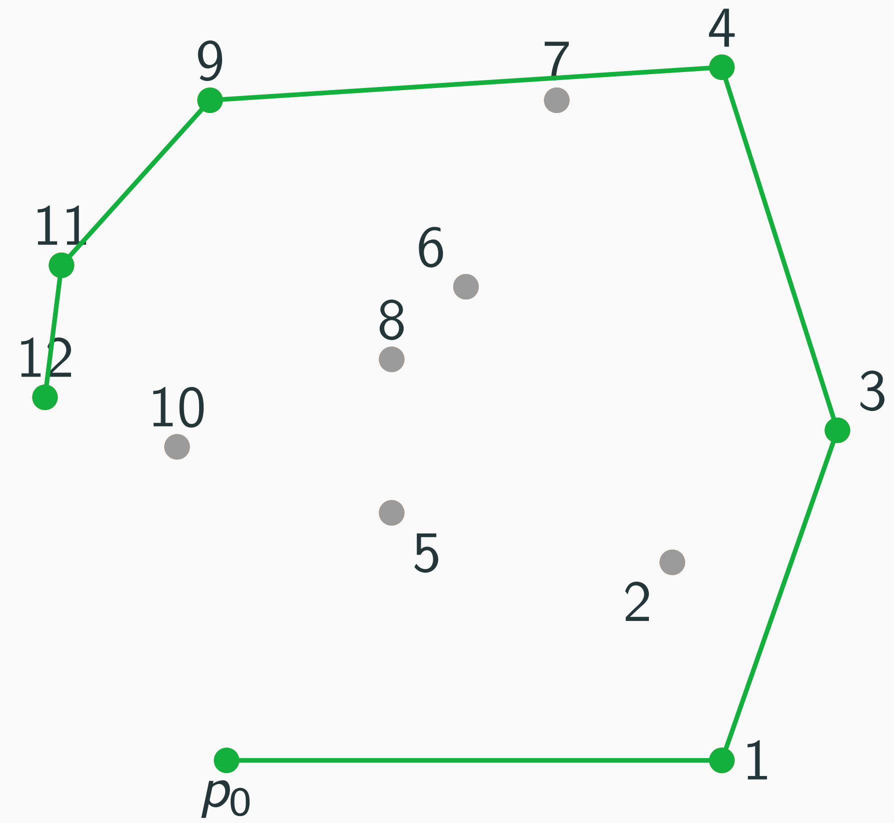

9
7
4
11
6
8
12
3
10
5
2
1
p_0



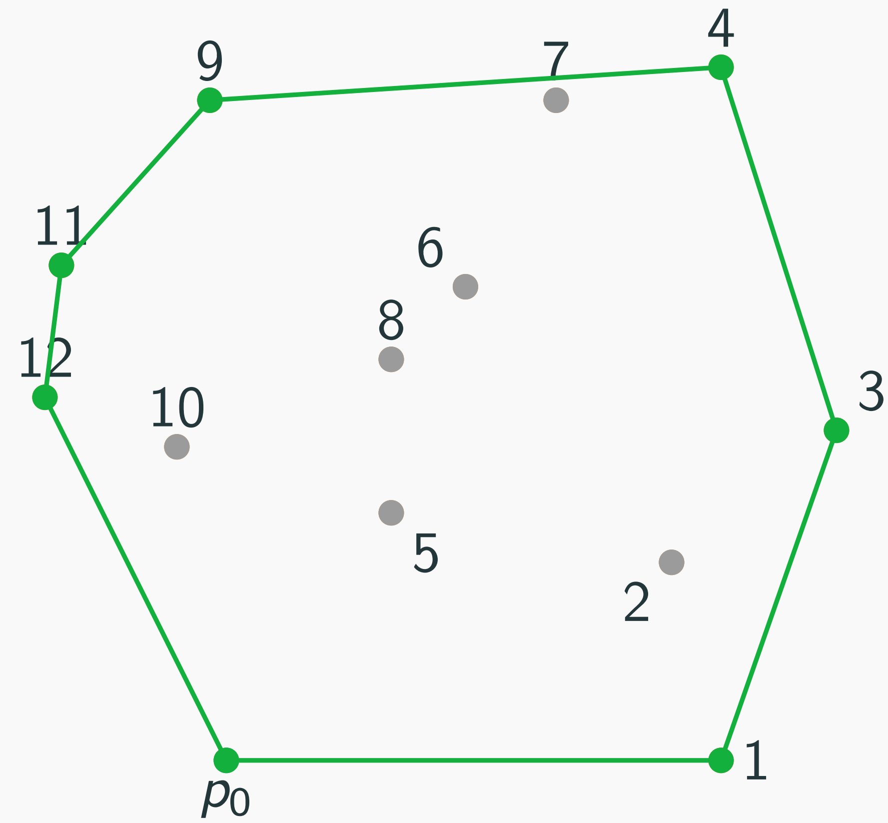
9
7
4
11
6
8
12
3
10
5
2
1
p_0



9
7
4
11
6
8
12
10
3
5
2
p_0
1



4
9
7
11
6
8
12
3
10
5
2
1
p_0



9
7
4
11
6
8
12
3
10
5
2
1
p_0

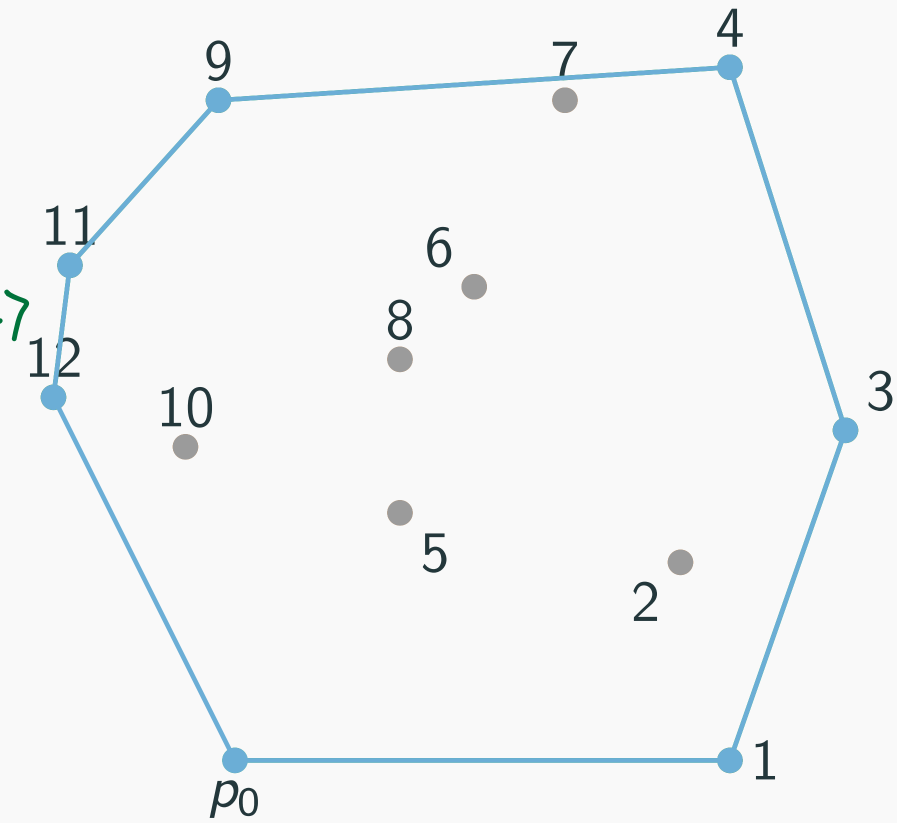
How do we represent a point in 2D?

(i) New data structure

```
struct point {
    double x, y;
}
```

(ii) use existing data type

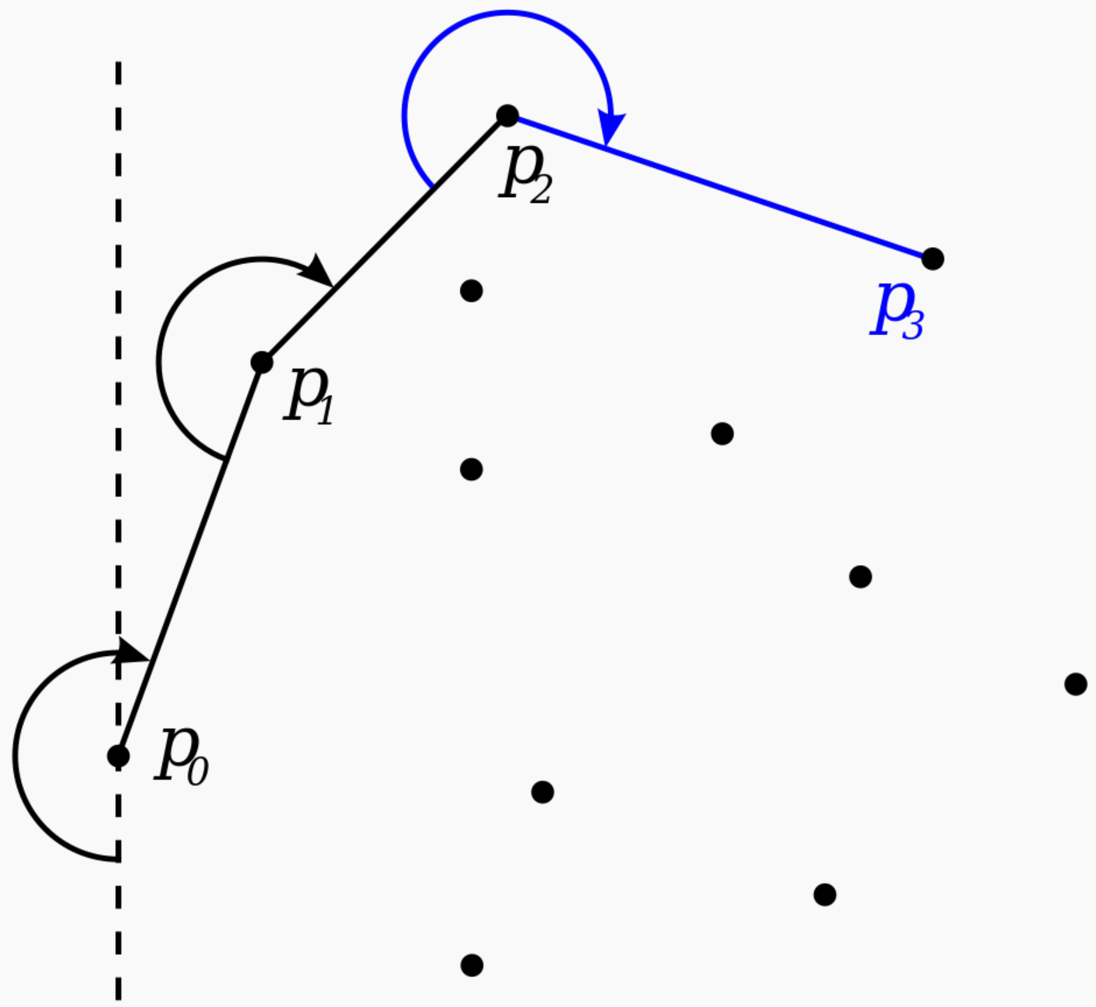
```
std::complex<double>
```



```
point hull[MAXN];
int convex_hull(vector<point> p) {
    int n = size(p), l = 0;
    sort(p.begin(), p.end(), cmp);
    for (int i = 0; i < n; i++) {
        if (i > 0 && p[i] == p[i - 1])
            continue;
        while (l >= 2 && ccw(hull[l - 2], hull[l - 1], p[i]) >= 0)
            l--;
        hull[l++] = p[i];
    }
    int r = l;
    for (int i = n - 2; i >= 0; i--) {
        if (p[i] == p[i + 1])
            continue;
        while (r - l >= 1 && ccw(hull[r - 2], hull[r - 1], p[i]) >= 0)
            r--;
        hull[r++] = p[i];
    }
    return l == 1 ? 1 : r - 1;
}
```

Complexity $O(nh)$ [i.e. worst case $O(n^2)$]

Gift Wrapping (Jarvis' algorithm)



Other algorithms for CH

Best Complexity Results

[Kirkpatrick, Seidel : 1986] $O(n \log h)$

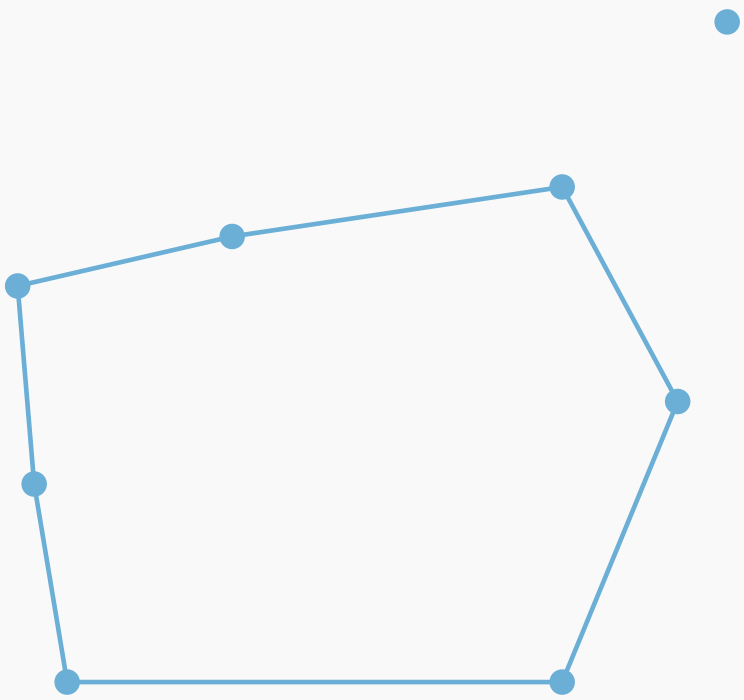
[Chan : 1996]

Many other algorithms exist

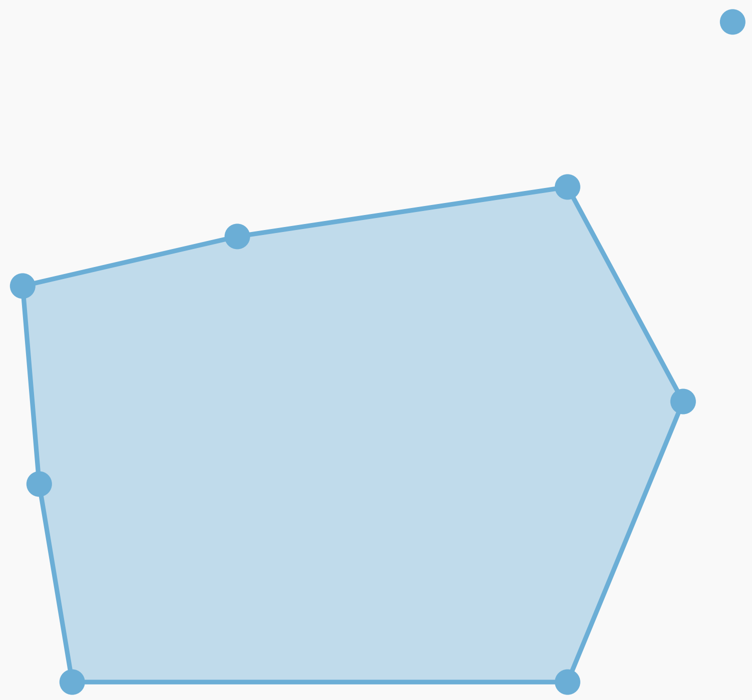
- Quick hull, similar idea to quicksort.
- Divide and conquer.

Some can be extended to three dimensions, or higher.

Is a point in a convex polygon?

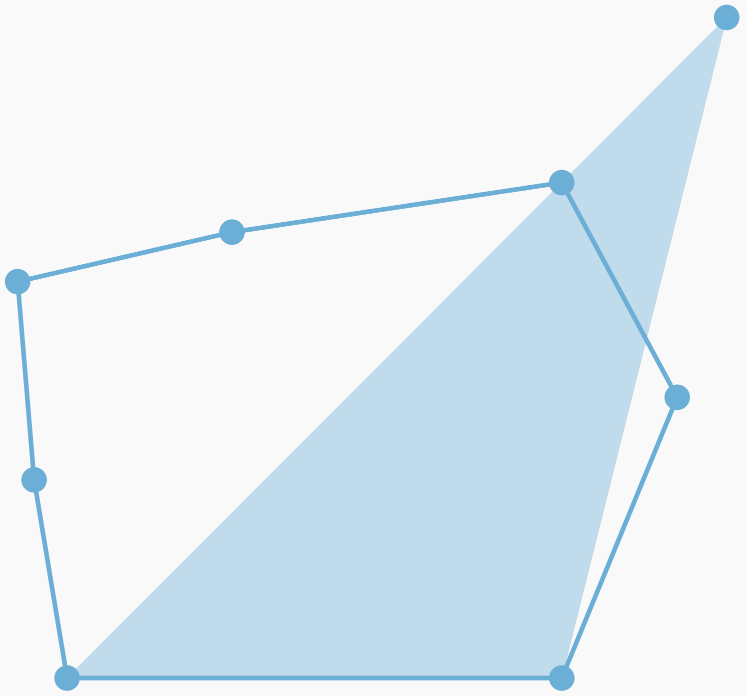


Is a point in a convex polygon?



- Calculate the area of the polygon.

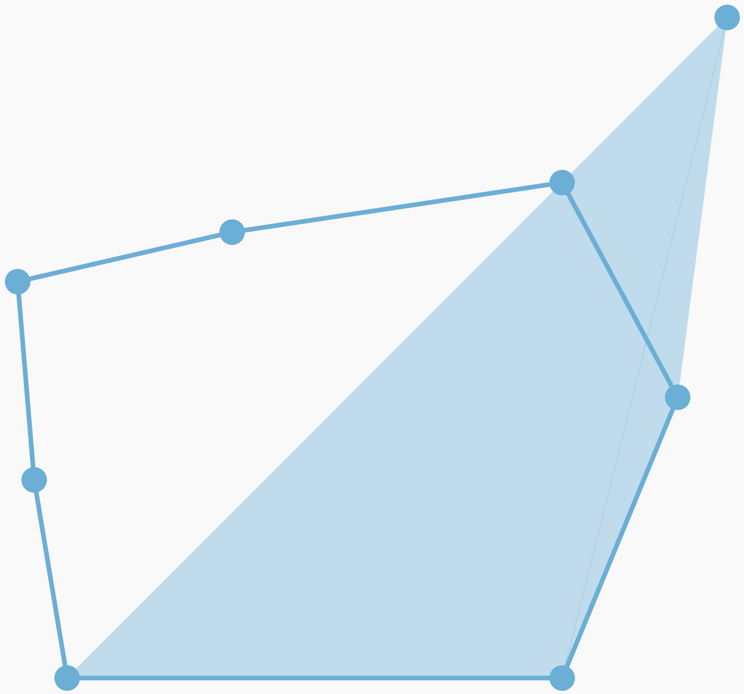
Is a point in a convex polygon?



- Calculate the area of the polygon.
- Sum the area of the triangles formed

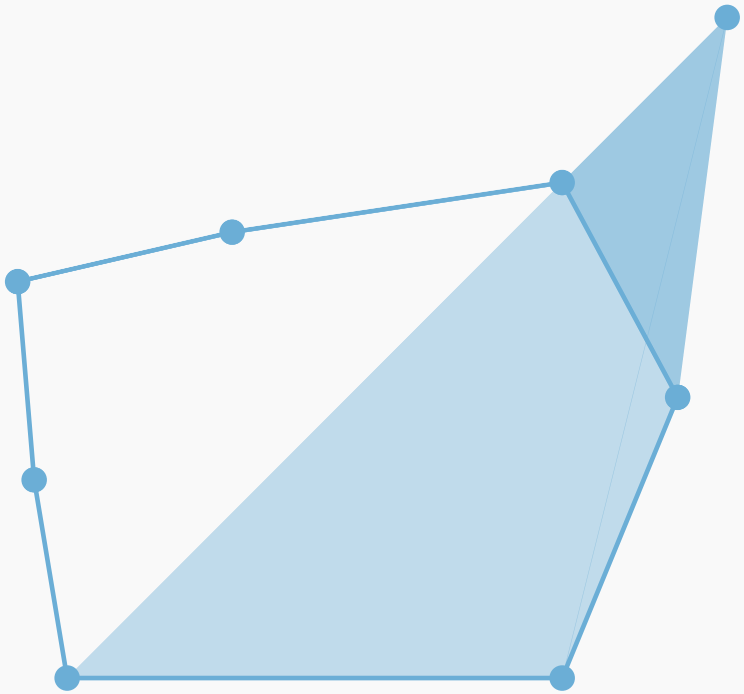
by the point and every two adjacent points.

Is a point in a convex polygon?



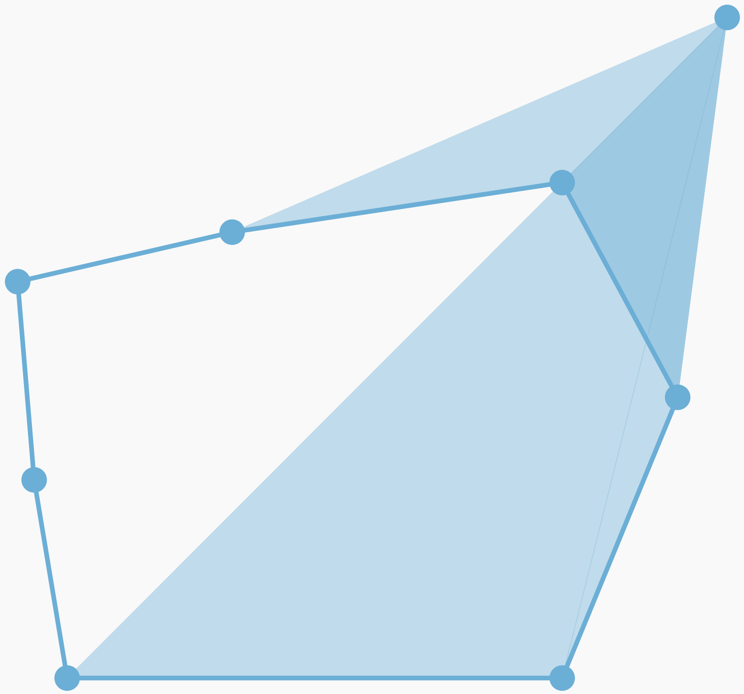
- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.

Is a point in a convex polygon?



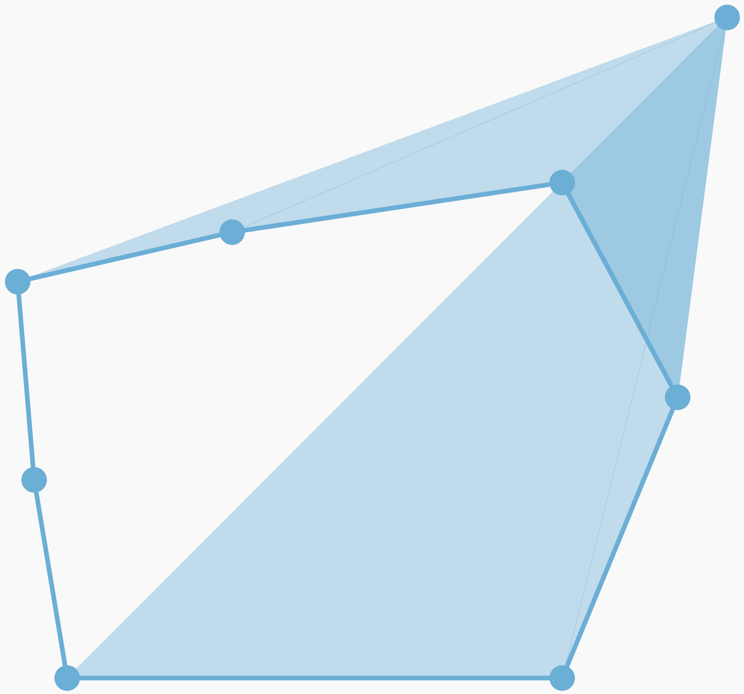
- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.

Is a point in a convex polygon?



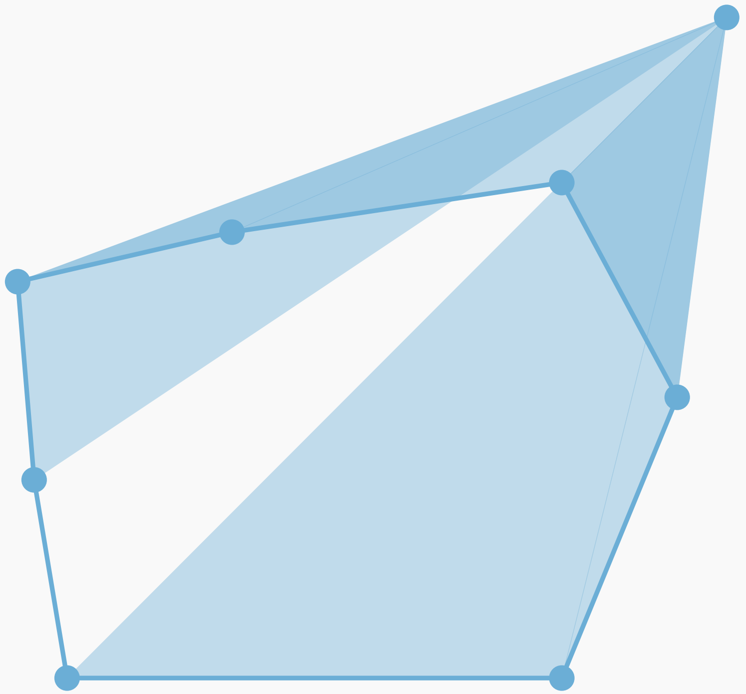
- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.

Is a point in a convex polygon?



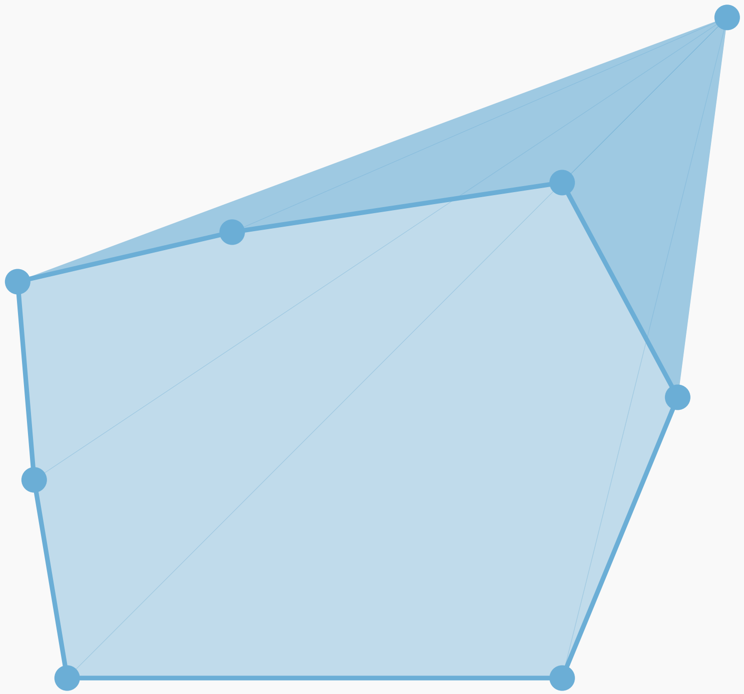
- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.

Is a point in a convex polygon?



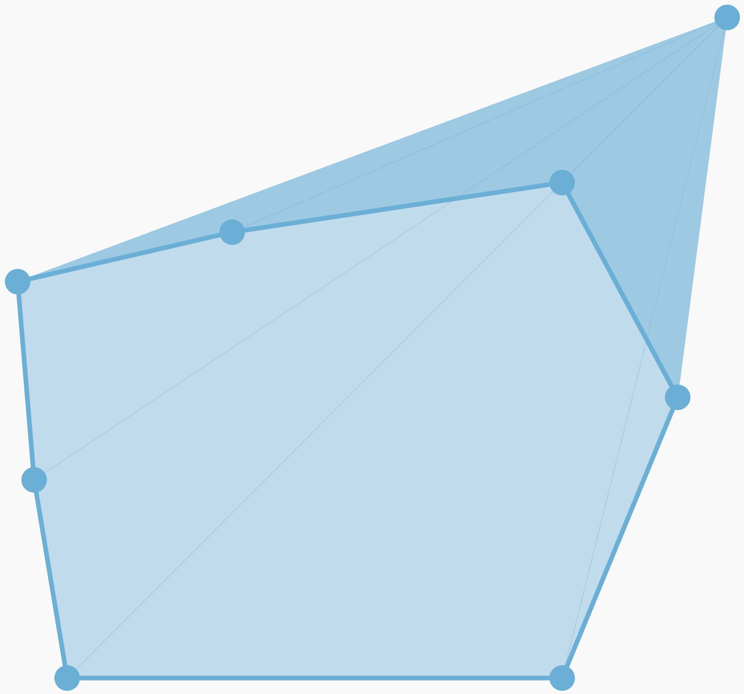
- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.

Is a point in a convex polygon?



- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.

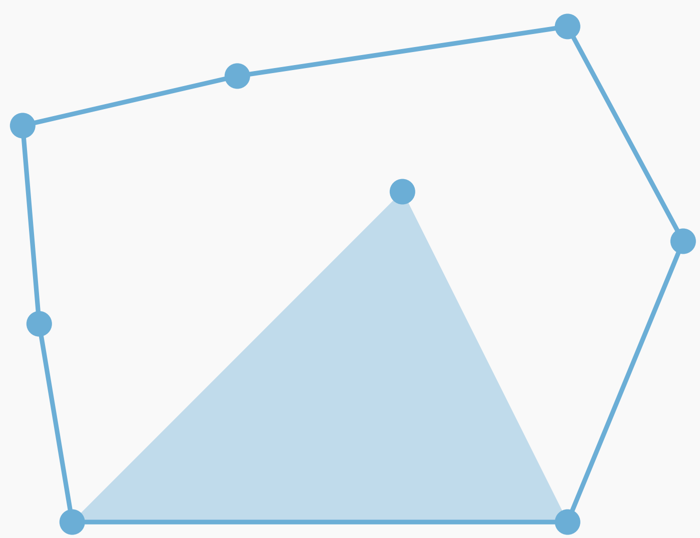
Is a point in a convex polygon?



- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.
- The total area of the triangles equals the area of the polygon iff the point is inside the polygon.

Is a point in a convex polygon?

(MEMBERSHIP oracle)



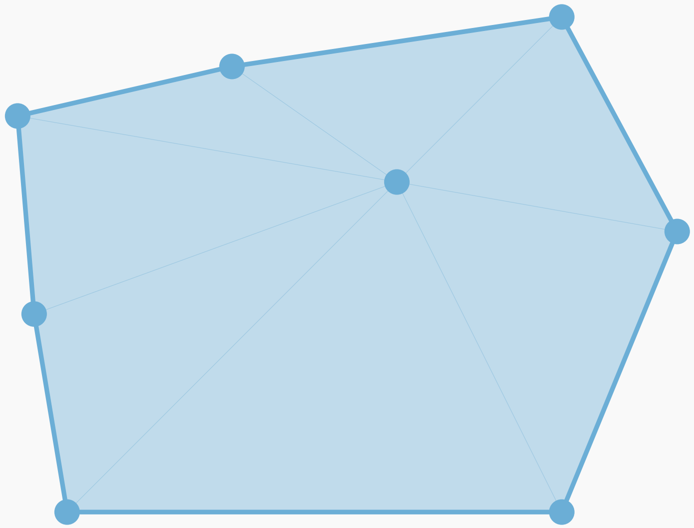
area of a triangle ≡ determinant computation ≡ CCW

- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.
- The total area of the triangles equals the area of the polygon iff the point is inside the polygon.

Is a point in a convex polygon?

TQ: What about in 3D?

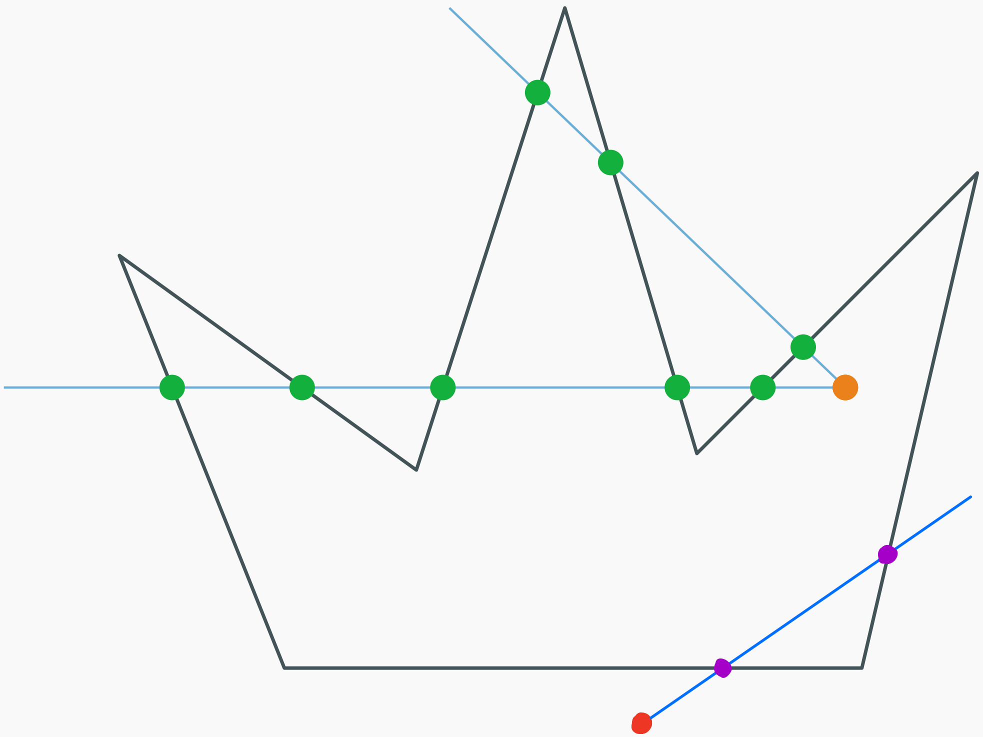
(MEMBERSHIP oracle)



- Calculate the area of the polygon.
- Sum the area of the triangles formed by the point and every two adjacent points.
- The total area of the triangles equals the area of the polygon iff the point is inside the polygon.

Is a point inside a non-convex polygon?

Q: How do we compute the intersections?



Odd-Even rule

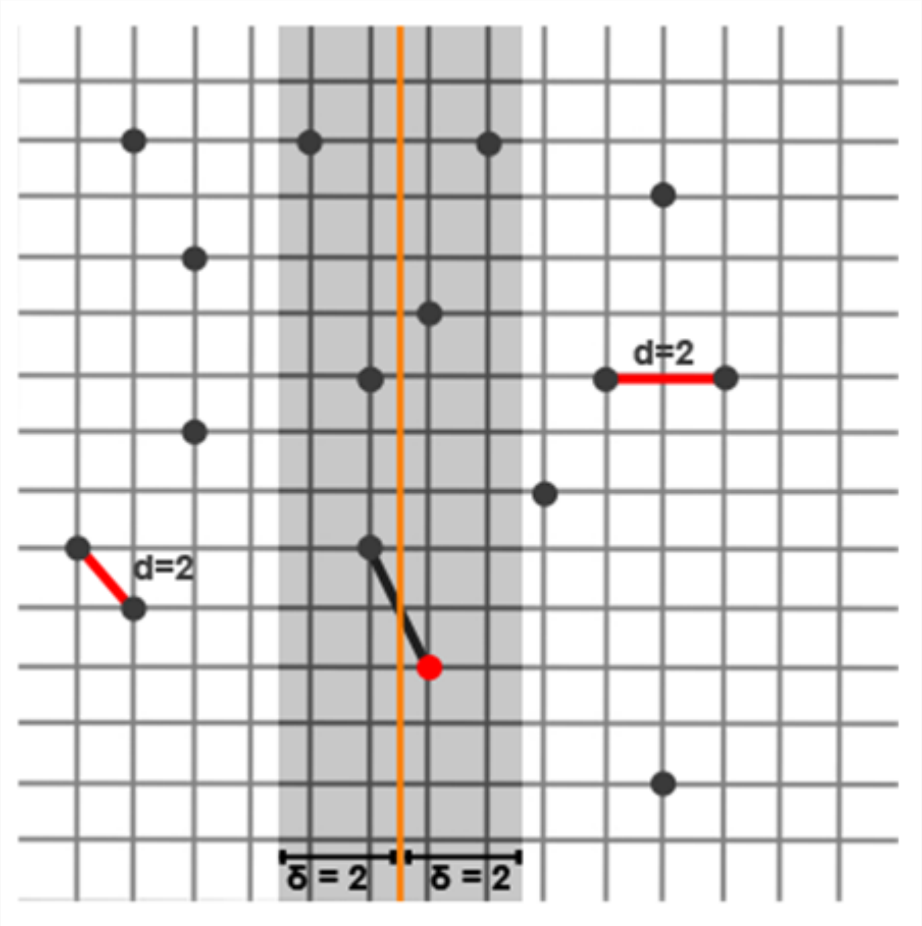
- Ray from the outside of the polygon to the point.
- Count the number of intersection points.
- If odd, then the point is inside the polygon.
- If even, then the point is outside the polygon.
- Does not matter which ray we pick.

Closest pair of points

Complexity : $O(n \lg n)$
worst case

- Sort points by the x-coordinate.
- Split the set into two equal sized sets by the vertical line of the medial x value.
- Solve the problem in the left and right subset.
- Let the smallest distance be δ. Create a strip in the middle of diameter 2δ.
- Find the minimum distance in the strip.

- Sort the points in the strip wrt y-coordinate
- In the strip, each point we need to compare it with at most 7 other



Binary Search

- We use *binary search* to find an element in a sorted array.
- But we can also use it in geometric problems:

Example

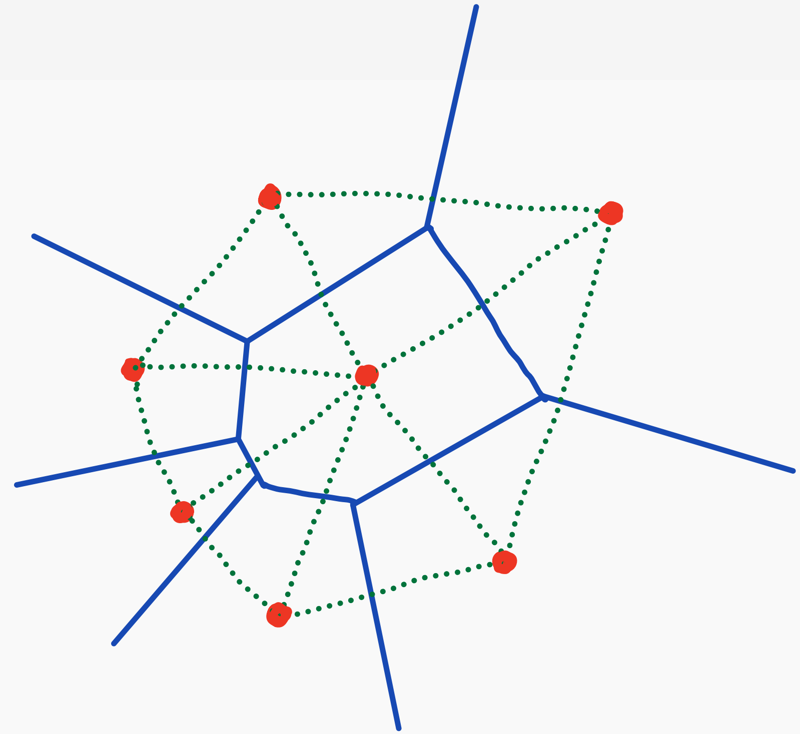
Find the largest inscribed circle in a polygon.

Example

Find the minimum point of a "convex" function f (ternary search)

- Initialize the search interval $[s, e]$
- Until $e - s$ becomes "small enough":
  - $m_1 := s + (e - s)/3$, $m_2 := e - (e - s)/3$
  - If $f(m_1) \leq f(m_2)$, set $e := m_2$
  - Otherwise, set $s := m_1$

Other algorithms



- Voronoi diagram
- Arrangements (Sweep Line alg.)
- Point location
- ...